

Step 7: Code Explanations for Model Experiments

Stephen Ebert

E-mail: stephenebert@gmail.com

ABSTRACT: These are detailed explanations of the code implementations used in step 7 of the capstone project for the Springboard bootcamp. The GitHub repository can be found [here](#).

Contents

1	00_paths_build_metadata.ipynb	1
2	01_retrieval_dataset.ipynb	3
3	01a_image_text_paired.ipynb	6
4	02_clip_baseline_training.ipynb	8
5	03_metrics_utils.ipynb	13
6	04_image_to_image_retrieval.ipynb	14
7	05_text_to_image.py	16
8	06_evaluation_visualization	19
9	07_dataset_to_dataset.ipynb	20
10	08_model_ablation.ipynb	20
11	09_visual_nn_demo	21
12	10_Run_Experiment	22

1 00_paths_build_metadata.ipynb

This notebook consolidates metadata from three datasets: Stable Diffusion, COCO, and Flickr30k into a single unified DataFrame and saves it as a **.parquet** file for downstream tasks. It begins by setting the data directory using environment variables, then reads and reformat captioned image data from each source. For Stable Diffusion, it loads image-prompt pairs from a CSV; for COCO, it extracts image-caption annotations from a JSON file using the `pycocotools` library; and for Flickr30k, it loads a CSV and renames relevant columns. Each record is enriched with a unique ID, domain tag, image path, and caption. Finally, the combined metadata is saved to disk for efficient access during model training and evaluation.

Now let's explain each segment of the code.

1. Import necessary modules:

- `Path` from `pathlib` is used for object-oriented file path handling.

- `os` is used for interacting with the operating system and environment variables.
 - `load_dotenv` from `dotenv` loads environment variables from a `.env` file.
 - `load_dotenv()` loads environment variables from a `.env` file into the runtime environment which makes them accessible through `os.getenv`
2. Now we need to establish our path to get our data. I have the data stored locally so I will use

```
1 DATA_ROOT = Path(os.getenv(
2     "DATA_ROOT",
3     r"C:\Users\steph\OneDrive\Desktop\Springboard\Springboard\Capstone\
    step2\data"
4 )).expanduser().resolve()
```

`DATA_ROOT` path does:

- Tries to get the value of the `DATA_ROOT` environment variable.
 - If it is not set, it falls back to a hardcoded Windows file path.
 - `Path(...).expanduser().resolve()` converts the string into a `Path` object, expands if present, and resolves it to an absolute path.
3. We want to check the directory actually exists or else we raise an `AssertionError` which is the purpose of
- ```
assert DATA_ROOT.exists(), f"DATA_ROOT not found"
```

1. We import the usual packages

- `pandas` for data manipulation
- `tqdm` so that we can track our code's evaluation with progress bars.
- `COCO` helper to load in annotations from the `COCO` dataset.
- `contextlib` and `os` are used to suppress `COCO`'s print statements.
- `meta_rows` list to accumulate all image-caption pair metadata

2. We load in our Stable Diffusion data with the `DATA_ROOT` path we established earlier:

```
1 sd_csv = DATA_ROOT / "SD" / "custom_prompts_df.csv"
2 df_sd = (pd.read_csv(sd_csv)
3 .rename(columns={"image_file": "image_id", "prompt": "caption"
4 }))
4 df_sd["image_id"] = df_sd["image_id"].str.replace(r"^images[/\\]", "",
 regex=True)
```

which reads the CSV of Stable Diffusion prompts and renames columns for consistency and cleans up file paths by removing `images/` prefix from filenames.

3. We load the `COCO` data

```

1 ann_json = DATA_ROOT / "coco" / "annotations" / "captions_train2017.
 json"
2
3 with open(os.devnull, "w") as fnull:
4 with contextlib.redirect_stdout(fnull):
5 coco = COCO(str(ann_json))

```

then loop over each annotation and append structured metadata (like Stable Diffusion) to `meta_rows`

```

1 for a in tqdm(coco.loadAnns(coco.getAnnIds()), desc="COCO"):
2 meta_rows.append(dict(
3 id = f"co_{a['id']}",
4 split = "train",
5 domain = "coco",
6 image_path = str(DATA_ROOT / "coco" / "train2017" / f"{int(a['
 image_id']):012d}.jpg"),
7 caption = a["caption"]))

```

We do the same thing with the Flickr30k data and save the files to a `.parquet` file by combining all the metadata into one `DataFrame`.

```

1 meta = pd.DataFrame(meta_rows)
2 out_pq = Path("../data/metadata.parquet")
3 out_pq.parent.mkdir(exist_ok=True)
4 meta.to_parquet(out_pq, index=False)
5 print(f"Wrote {len(meta):,} rows -> {out_pq}")

```

## 2 01\_retrieval\_dataset.ipynb

This code defines a custom PyTorch `Dataset` class called `RetrievalDataset` designed for use in a cross-modal image-text retrieval task. It loads metadata from a `.parquet` file containing image paths and their associated captions, filters it by split (defaulting to “train”), and prepares the data for training or evaluation. Each image is preprocessed using standard CLIP normalization and resizing transformations, while each caption is tokenized using OpenCLIP’s `SimpleTokenizer`. The `__getitem__` method returns a dictionary containing the processed image tensor, tokenized text, domain label, and sample ID. This setup enables easy batching and iteration over multimodal data, facilitating training or testing of models like CLIP that align images and text.

Let’s explain some of the code

### 1. Some noteworthy packages

- `torch` and `Dataset` are foundational for PyTorch’s data loading framework which I use here.
- `transforms` from `torchvision` are used to define image preprocessing steps.
- `PIL.Image` is to open and manipulate images.

- `clip_tokenizer` imports the tokenizer used by [OpenCLIP](#) to process text captions.
2. We first need to define a transform pipeline to ensure all images are consistently sized and normalized which will be fed into a CLIP model for tasks like image-text matching or retrieval. The code below for our transformer does just that

```

1 transform = T.Compose([
2 T.Resize(224, antialias=True),
3 T.CenterCrop(224),
4 T.ToTensor(),
5 T.Normalize((0.48145466, 0.4578275, 0.40821073),
6 (0.26862954, 0.26130258, 0.27577711)),
7])

```

- `T.Resize(224, antialias = True)` resizes the smaller image edge to 224 pixels while maintaining aspect ratio. `antialias=True` improves quality during downsampling.
- `T.CenterCrop(224)` crops the center  $224 \times 224$  pixel region to ensure a fixed input size.
- `T.ToTensor()` Converts the PIL image to a PyTorch tensor and scales pixel values to the  $[0, 1]$  range.
- `T.Normalize(mean, std)` normalizes the tensor using CLIP's specific mean  $\mu$  and standard deviation  $\sigma$  values per color channel (RGB). This aligns input distributions with what CLIP expects during inference or training.

We can see this setup ensures all images are consistently sized and normalized so they can be fed into a CLIP model for tasks like image-text matching or retrieval.

3. Next we use a custom PyTorch `Dataset` class called `RetrievalDataset` which prepares and serves the image-caption pairs for training or evaluating models like CLIP. This code will be part of a class called `RetrievalDataset(Dataset)` which inherits from `torch.utils.data.Dataset` so that it will be used with PyTorch data loaders. We send in our `parquet_path`, `split` (training/validation subset data), and `transform` (image preprocessing pipeline)

```

1 def __init__(self, parquet_path=, split="train", transform=transform):

```

We also have

- loads the dataset metadata from the file and filters it by the specified split.
- ```

1 self.df = pd.read_parquet(parquet_path)
2 self.df = self.df[self.df.split == split].reset_index(drop=True)

```
- Convert captions into token sequences with OpenCLIP's `SimpleTokenizer` and stores the transform pipeline for preprocessing images

```

1 self.tok = clip_tokenizer.SimpleTokenizer()
2 self.transform = transform

```

- To get the dataset length (returns the number of samples in the dataset) and getting the item

```

1 def __len__(self): return len(self.df)
2 def __getitem__(self, idx): ...

```

- The dunder method `__getitem__(self, idx)` first fetches one sample by index

```
1 r = self.df.iloc[idx]
```

then retrieves the row of metadata

```
1 img = self.transform(Image.open(r.image_path).convert("RGB"))
```

which opens the image, converts it to RGB and apply the preprocessing transforms to tokenize the associated caption using OpenCLIP’s tokenizer

```
1 txt = self.tok(r.caption)
```

and returns a dictionary with the processed image, tokenized text, the domain (e.g., “coco”, “sd”, or “flickr”), and a unique identifier

```
1 return {"image": img, "text": txt, "domain": r.domain, "id": r.id}
```

We can see this works as intended when loading the `RetrievalDataset`

```

In [7]: 1 ds = RetrievalDataset()
        2 print("Dataset size:", len(ds))
        3 ex = ds[0]
        4 ex["image"].shape, ex["text"][:10], ex["domain"]

Dataset size: 850668

Out[7]: (torch.Size([3, 224, 224]),
        tensor([[49406, 3086, 539, 674, 5508, 20414, 9920, 550, 17143, 49407,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]),
        'sd')

```

The printed result shows three key outputs for the first sample in the dataset

- `torch.Size([3, 224, 224])` indicates the shape of the preprocessed image. It has 3 color channels (RGB), and has been resized and cropped to a 224×224 pixel square exactly as specified by the transform pipeline.
- `tensor([...])` is the tokenized caption returned as a tensor of token IDs using OpenCLIP’s `SimpleTokenizer`.
- ‘sd’ shows the domain label for the sample, which in this case is ‘sd’, meaning it came from the Stable Diffusion dataset.

- The `Dataset` size 850668 confirms that the dataset contains 850,668 samples after filtering by the “train” split. This setup confirms the dataset class is correctly handling images and captions from a large, mixed-source dataset for use in training a retrieval model.

3 01a_image_text_paired.ipynb

This notebook evaluates image-to-text retrieval performance by computing Recall@k metrics using precomputed image and text embeddings. It begins by loading the image (`img_embs.npy`) and text (`txt_embs.npy`) embedding arrays, along with their corresponding IDs (`ids.json`) from a specified experiment directory. It also loads the metadata from a `.parquet` file and aligns it with the embeddings using those IDs. Both image and text embeddings are L2-normalized to ensure cosine similarity can be computed efficiently via dot product. The core function `image_to_text_recall` divides the dataset into chunks for memory efficiency, calculates similarity scores between image and text embeddings, retrieves the top-k most similar text entries for each image, and checks whether the correct caption is among them. The recall is averaged over all images and returned as a percentage. Recall@1, @5, and @10 are computed, stored in a dictionary, printed to the console, and saved as a JSON file for later analysis. This process quantifies how accurately the model retrieves matching captions given images.

1. We load embeddings and metadata

```
1 RUN_DIR = Path("experiments/RN50_20250623_214602")
2 IMG_EMB = np.load(RUN_DIR / "img_embs.npy").astype("float32")
3 TXT_EMB = np.load(RUN_DIR / "txt_embs.npy").astype("float32")
4 IDS = json.loads((RUN_DIR / "ids.json").read_text())
5 PARQUET = Path("C:/Users/steph/OneDrive/Desktop/data/metadata.parquet")
6 META = pd.read_parquet(PARQUET).set_index("id").loc[IDS]
```

- `RUN_DIR` is the directory containing the output of a model run (e.g., using ResNet-50 features).
- `IMG_EMB` and `TXT_EMB` are NumPy arrays containing image and text embeddings, respectively.
- `IDS` contain the list of unique IDs corresponding to each sample, linking embeddings to metadata.
- Loads the original metadata and filters it to match the order and subset of embeddings based on the `IDS` list.

2. We normalize embeddings

```
1 img_norm = IMG_EMB / norm(IMG_EMB, axis=1, keepdims=True)
2 txt_norm = TXT_EMB / norm(TXT_EMB, axis=1, keepdims=True)
```

We apply L2 normalization to the image and text embeddings so that their dot product can be interpreted as cosine similarity $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{ab}$ for two vectors $\vec{a}, \vec{b}$ with magnitudes a, b . This normalization ensures fair similarity comparison regardless of vector magnitude.

3. We use the function `image_to_text_recall(k, chunk)` to calculate the Recall@k metric for an image-to-text retrieval task, which measures how often the correct caption is found in the top-k most similar text embeddings for each image after processing `chunk` of data to reduce memory usage. Let's talk about its contents

- For n total number of image embeddings and `hits` (a boolean array to record whether each image successfully retrieved its matching text within the top-k) are given by

```
1 n = img_norm.shape[0]
2 hits = np.zeros(n, dtype=bool)
```

- Then we loop over the image embeddings e in segments of size `chunk` for efficiency

```
1 for s in range(0, n, chunk):
2     e = min(s + chunk, n)
```

where the min is necessary for handling the last chunk when dividing a sequence into smaller, fixed-size chunks which ensures that the slice always stays within the valid bounds of the original data.

- We compute the cosine similarities between the chunk of image embeddings and all text embeddings using matrix multiplication (`@`), since the embeddings are L2-normalized.

```
1 sim = img_norm[s:e] @ txt_norm.T
```

then we get the top-k matches by using the `argpartition` to get indices of the top-k most similar text embeddings for each image in the chunk

```
1 topk = np.argpartition(-sim, k - 1, axis=1)[: , :k]
```

- We check whether the correct caption at the same index position is present in the top-k predictions for each image

```
1 rows = np.arange(s, e)[: , None]
2 hits[s:e] = np.any(topk == rows, axis=1)
```

and computes the final recall which is the percentage of images for the correct caption was retrieved in the top-k results

```
1 return hits.mean() * 100
```

This function quantifies how well the model retrieves the correct caption for each image. A higher Recall@k score indicates better performance, especially in aligning visual and textual representations.

4. We make the function call `image_to_text_recall(k)` which computes how often the correct caption is among the top-k text matches for each image and save results to a JSON

```
1 # Generate Recall@k for k=1,5,10
2 metrics_it = {f"R@{k}": round(image_to_text_recall(k), 2) for k in (1,
    5, 10)}
3 print(metrics_it)
4 # Save metrics to JSON
5 out_file = RUN_DIR / "image_text_metrics.json"
6 json.dump(metrics_it, open(out_file, "w"), indent=2)
7 print("Saved:", out_file, metrics_it)
```

The output is `{‘R@1’: 53.25, ‘R@5’: 78.4, ‘R@10’: 88.05}` and summarizes the image-to-text retrieval performance of the model using the Recall@k metric. A Recall@1 of 53.25% means that in over half of the cases, the correct caption was ranked as the most similar to its corresponding image. Recall@5 and Recall@10 values of 78.4% and 88.05%, respectively, indicate that the correct caption was found among the top 5 and top 10 retrieved results in the vast majority of cases. These results suggest the model is highly effective at aligning visual and textual information, with strong semantic consistency between image and caption embeddings.

4 02_clip_baseline_training.ipynb

This notebook evaluates and visualizes the performance of an image-text retrieval model using different dataset sizes and CLIP-style architectures. It begins by setting configuration presets (e.g., model type, device, batch size) and selecting the appropriate one based on the specified PRESET (e.g., “cpu-fast”). It then loads a metadata `.parquet` file containing image-caption pairs and optionally limits the sample size. Using PyTorch and OpenCLIP, the code loads the corresponding model (e.g., RN50 or ViT-B-32), tokenizes the captions, preprocesses the images, and computes L2-normalized embeddings for both modalities in batches.¹ These embeddings, along with the sample IDs, are saved to disk.

Next, it calculates retrieval metrics including Recall@1, Recall@5, Recall@10, and the median rank by computing cosine similarity between the text and image embeddings. These metrics are saved in JSON format, along with the run configuration for reproducibility.

Finally, the notebook loads results from all past runs in the `experiments` directory, extracts key metrics and sample sizes, and visualizes how retrieval performance scales with dataset size using `Matplotlib`. It plots recall scores against the number of samples used

¹Here are some comments on these networks. Note that RN50 refers to ResNet-50 and is a residual network with 50 convolutional layers. ResNet architectures, including ResNet-50, were developed to solve the vanishing gradient problem. As neural networks get deeper, the gradients and updating the weights can become extremely small, making it difficult for the network to learn. ViT-B-32, often as part of larger models like CLIP (Contrastive Language-Image Pre-training) by OpenAI, is a vision transformer with 32×32 pixel patches. Instead of using convolutional layers, ViTs directly apply the Transformer architecture which revolutionized NLP with models like BERT and GPT to images. Unlike CNNs that process images locally and build up global understanding hierarchically, Transformers process all parts of the image simultaneously through self-attention, allowing them to capture global dependencies directly from the start.

and saves this comparison as `scaling_plot.png`, providing valuable insight into how model accuracy improves with more data.

1. This block of code defines configuration variables that control how the image-text retrieval experiment will run how the image-text retrieval experiment will run. We specify the file path to the metadata file in Parquet format that contains image paths, captions, IDs, and domains for the dataset

```
1 PARQUET = r"C:/Users/steph/OneDrive/Desktop/data/metadata.parquet"
```

We then select a predefined configuration for the experiment. For our case, “`cpu-fast`” indicates the code will run on the CPU using a model optimized for faster execution (e.g., RN50 with moderate batch size and sample limits). In the code, we also have `BATCH_OVERRIDE` to allow manual override of the batch size where `None` means it will use the batch size defined by the selected preset. For this example, we set a 2000 limit for `MAX_SAMPLES_OVERRIDE` for fast testing. We also have

```
1 MODEL_OVERRIDE = None
2 PRETRAINED_OVERRIDE = None
```

where the user manually specify a different model architecture (e.g., “RN50”, “ViT-B-32”). `None` means the code will use the model specified by the preset. The pre-trained line lets the user override the default pretrained weights source (e.g., “openai”, “laion2b” etc). Since it is `None`, it will stick to the preset’s pretrained model. These variables are parameters that let the user control the experiment’s dataset, model, and execution settings without changing the main code logic. They provide flexibility for experimenting with different setups.

2. We want to set up the environment, configuration, and device for running a cross-modal image-text retrieval experiment. Let’s break it down:

- We add the project source path with

```
1 sys.path.append(r"C:/Users/steph/OneDrive/Desktop/cross-modal -
    retrieval/src")
2 from retrieval import RetrievalDataset
3 repo_root = Path.cwd()
4 if not (repo_root / "src").exists():
5     repo_root = repo_root.parent
6 sys.path.append(str(repo_root / "src"))
```

which adds the custom project’s `src` directory to Python’s module search path so it can import `RetrievalDataset` then dynamically ensures the `src` directory is on the Python path regardless of where the script is run.

3. Next we define the preset configurations which schematically look like this

```
1 PRESETS = {
2     "gpu": {...},
3     "cpu-fast": {...},
```

```

4     "cpu-tiny": {...},
5 }

```

where `gpu` uses a large model (ViT-B-32) on GPU with a bigger batch size, `cpu-fast` uses RN50 on CPU with a moderate sample limit of 50,000 and `cpu-tiny` uses RN50 on CPU with a smaller sample limit of 10,000. Once we have the configuration, we make a copy via `cfg = PRESETS[PRESET].copy()` so that it can be directly modified

4. We do the device selection which checks the CUDA (GPU) is available. If the preset requested GPU but none is available, it switches to CPU and informs the user

```

1 device = cfg["device"]
2 if device == "cuda" and not torch.cuda.is_available():
3     print("CUDA not available - switching to CPU")
4     device = "cpu"

```

In summary, this code prepares the environment by importing libraries, setting up paths, loading configuration presets, and choosing the appropriate device (CPU or GPU) for running a cross-modal retrieval experiment. It ensures flexibility and portability so experiments can run on different hardware and with different model setups.

5. This next code sets up the directory, dataset, data loader, and model for an experiment run. It ensures that everything is properly configured to start generating embeddings and computing retrieval metrics. Each run is stored in its own timestamped folder for easy tracking and reproducibility.

- We create a unique experiment directory with a timestamp using `datetime`, make a path `experiments/RN50_20250627_153210`, where RN50 is the selected model in the configuration. We then create the dictionary if it doesn't exist and print the absolute path for confirmation. After this, we load in the full dataset containing image-caption pairs from the metadata parquet file

```

1 ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
2 exp_dir = Path("experiments") / f"{cfg['model']}_{ts}"
3 exp_dir.mkdir(parents=True, exist_ok=True)
4 print("Experiment dir:", exp_dir.resolve())
5 full_ds = RetrievalDataset(PARQUET)

```

- If a sample limit (`max_samples`) is fixed, it randomly selects that many indices to create a smaller dataset subset else it uses the full dataset

```

1 if cfg["max_samples"]:
2     rand_idx = torch.randperm(len(full_ds))[: cfg["max_samples"]].tolist()
3     dataset = Subset(full_ds, rand_idx)
4 else:
5     dataset = full_ds

```

- Finally, we create a data loader and load the model into it. The snippet of code does just that

```

1 loader = DataLoader(
2     dataset,
3     batch_size=cfg["batch"],
4     shuffle=False,
5     num_workers=4,
6     pin_memory=(device == "cuda"),
7 )
8 model, _, _ = open_clip.create_model_and_transforms(
9     cfg["model"], pretrained=cfg["pretrained"], device=device
10 )
11 model.eval()
12 print(f"Model {cfg['model']} ({cfg['pretrained']}) on {device}")

```

Here we wrap the dataset in a PyTorch `DataLoader` for efficient batch processing with four worker threads to load data in parallel and enable `pin_memory` for faster GPU data transfer if using CUDA. Finally, we load the specified model, like RN50 or ViT-B-32 with pretrained weights such as “openai” or “laion2b_s34b_b79k” onto the specified CPU or GPU.

6. The next piece of code efficiently generates and saves normalized image and text embeddings for the entire dataset. These embeddings are later used for computing similarity scores and evaluating retrieval performance.

- For starters, we define empty arrays to accumulate image embeddings, text embeddings, and corresponding IDs across all batches with `img_parts`, `txt_parts`, `ids = [], [], []` to then loop through a data loader

```

1 with torch.no_grad():
2     for batch in tqdm(loader, desc="Embedding", mininterval=1):

```

where, for convenience to save memory and speed up inference, we turned off gradient calculation and iterate over batches while using a progress bar labeled “Embedding.”

- Finally, we process each batch of information, concatenate, and save it. The processing involves

```

1 imgs = batch["image"].to(device, non_blocking=True)
2 caps = tokenize(batch["caption"]).to(device, non_blocking=True)
3 ie = model.encode_image(imgs)
4 ie = ie / ie.norm(dim=-1, keepdim=True)
5 te = model.encode_text(caps)
6 te = te / te.norm(dim=-1, keepdim=True)
7 img_parts.append(ie.cpu())
8 txt_parts.append(te.cpu())
9 ids.extend(batch["id"])
10 np.save(exp_dir / "img_embs.npy", img_arr)
11 np.save(exp_dir / "txt_embs.npy", txt_arr)
12 json.dump(ids, open(exp_dir / "ids.json", "w"))
13 print("Embeddings saved")

```

Here we move images and tokenized captions to the CPU or GPU and use `non_blocking = True` for faster data transfer when using CUDA. The `ie = ...` computes image embeddings and L2-normalizes them so that the dot product equals cosine similarity. `dim = -1` tells PyTorch to calculate the norm along the last dimension (e.g., this means it will calculate the L2 norm for each of the n -dimensional embedding vectors with p images independently). `keepdim=True` ensures the dimension along which the norm is computed is of unit size. Then we compute the normalized text embeddings in the similar manner as we did with the image embedding. We then move these embeddings to CPU and store them along with IDs.

7. This next block of code computes retrieval metrics (Recall@k and median rank) for the generated image-text embeddings, saves the results, and reports how long the process took.

- We compute the cosine similarity matrix between text and image embeddings `sim = txt_arr @ img_arr.T`. Since embeddings were L2-normalized earlier, this dot product gives cosine similarity and `sim[i, j]` represents how well text i matches with image j .
- We define Recall@k function to get the top- k most similar images for each text embedding, check if the correct image (same index) appears in the top- k for each text, and return the proportion of correct matches

```
1 def r_at_k(mat, k):
2     topk = np.argpartition(-mat, k - 1, axis=1)[: , :k]
3     return float(np.mean(np.any(topk == np.arange(len(mat))[: ,
4     None], axis=1)))
5 metrics = {f"R@{k}": round(r_at_k(sim, k) * 100, 2) for k in (1,
6     5, 10)}
```

8. Another metric is the *median rank* which calculates and stores the median rank across all queries

```
1 ranks = np.argsort(-sim, axis=1)
2 med = [int(np.where(ranks[i] == i)[0][0] + 1) for i in range(len(sim))]
3 metrics["median_rank"] = int(np.median(med))
```

which fully sorts the similarity scores for each text query and determine the rank position of the correct image for each text.

9. Output Metrics: {'R@1':48.35, 'R@5':76.55, 'R@10':85.2, 'median_rank':2}:

The output indicates that the image-text retrieval model performed well in aligning images and captions. Specifically, the model achieved a Recall@1 of 48.35%, meaning that in nearly half of the cases, the correct image was retrieved as the most similar match for its corresponding caption. The Recall@5 and Recall@10 scores were 76.55% and 85.2%, respectively, showing that the correct image appeared within the top 5 and top 10 retrieved results in the majority of cases. Additionally, the median rank

of 2 means that for half of the queries, the correct image ranked second or better. The entire process of generating embeddings and computing these metrics completed in 5.4 minutes, demonstrating both strong performance and efficient execution. We plot the results here

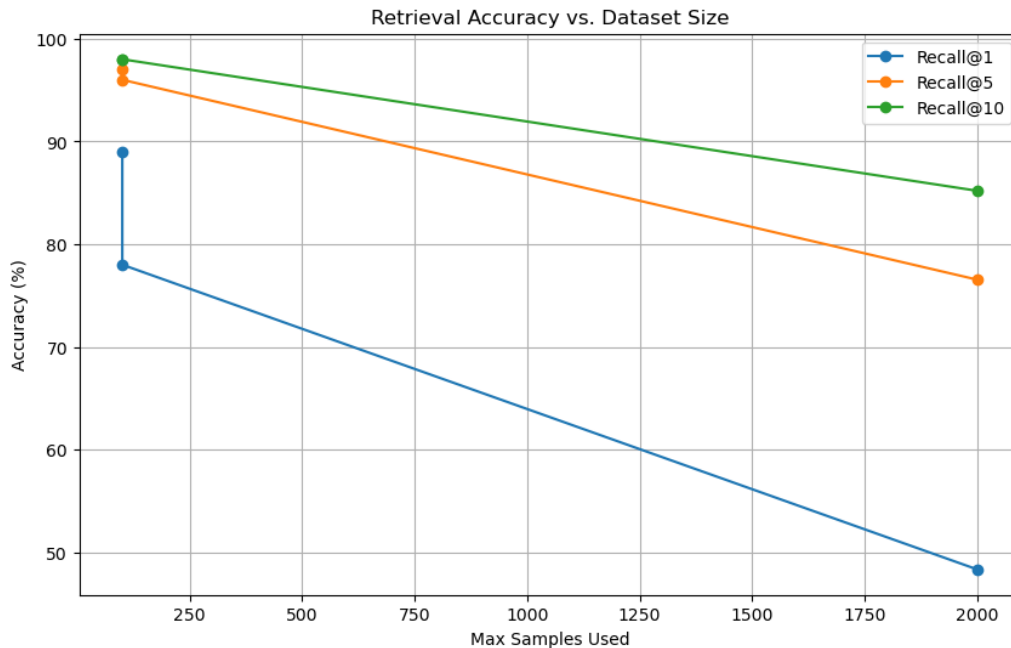


Figure 1. Retrieval Accuracy vs. Dataset Size. The chart illustrates how retrieval accuracy (Recall@1, Recall@5, and Recall@10) changes as the number of maximum samples used in the dataset increases.

5 03_metrics_utils.ipynb

This notebook defines functions and a workflow to compute, load, and visualize retrieval metrics for image-text experiments. The `02_clip_baseline_training.ipynb` notebook focuses on generating image and text embeddings using a baseline CLIP model. It sets up data loaders, runs the model on the dataset, and saves the resulting embeddings, along with configuration details and preliminary metrics. In contrast, the `03_metrics_utils.ipynb` notebook provides utilities and workflows for evaluating these embeddings. It defines functions to compute retrieval metrics such as Recall@K and median rank, both exactly and in a memory-efficient chunked manner. Additionally, it loads experiment outputs, aggregates results across runs, and visualizes performance (e.g., plotting Recall@1 across experiments). Together, these notebooks form a complete pipeline: one for generating model outputs, and the other for analyzing and interpreting retrieval performance.

The `recall_at_k` function calculates the exact Recall@K from a full similarity matrix by checking if the correct image (same index as the caption) appears in the top-k predictions. The `median_rank` function computes the median position where the correct image appears

for all captions. The `chunked_metrics` function is a memory-efficient alternative that computes Recall@K and median rank without forming the full similarity matrix at once; it processes text embeddings in chunks and accumulates results. The workflow then loads existing experiment metrics and configurations from `metrics.json` and `config.json` files in the experiments directory, assembles them into a DataFrame, and plots a bar chart showing Recall@1 for each experiment folder. Finally, it demonstrates how to reload embeddings from a specific run, recompute the metrics using `chunked_metrics`, and print the updated results. This setup allows both efficient evaluation of large datasets and easy tracking and visualization of retrieval experiment performance over time.

6 04_image_to_image_retrieval.ipynb

This notebook performs image-to-image retrieval and evaluates cross-domain retrieval performance. It begins by L2-normalizing all image embeddings so that cosine similarity can be efficiently computed using dot products. The `query_image` function retrieves the top- k most similar images for a given query, excluding the query image itself, and returns their indices and similarity scores. The `show_retrieval` function displays the query image alongside its top matches, showing their domains and similarity values to provide qualitative insight into retrieval results. The code runs this process for three random query images. The `image_to_image_recall` function computes cross-domain Recall@K by verifying whether at least one of the top- k matches for each image comes from a different domain. It processes embeddings in chunks for memory efficiency and masks self-similarity by setting diagonal similarities to negative infinity. Finally, the script prints Recall@1, Recall@5, and Recall@10 scores, saves the results to a JSON file, and reports the save location. This workflow evaluates how well the model retrieves visually similar images across domains.

1. This part of the code, `def show_retrieval(query_idx, k=5)` function, is designed to visualize and report image-to-image retrieval results for a given query image. This function allows one to visually inspect how well the model retrieves similar images. It shows both the query and the top matching results, along with their domains and similarity scores, helping assess whether the retrieval results make sense and whether the model captures meaningful visual similarity across domains. Here `query_idx` is the index of the image in the dataset to use as the query and `k` are the number of nearest neighbors to display.

- We display the query image

```
1 q_row = META.iloc[query_idx]
2 print("Query image domain:", q_row.domain)
3 display(Image.open(q_row.image_path).resize((224,224)))
```

from the metadata of the query image and print the domain (`coco`, `sd`, `flickr`) to then find nearest neighbors by calling the `query_image` (e.g., returns the indices and similarity scores of these nearest neighbors)

```
1 nn_idx, nn_score = query_image(query_idx, k)
```

- Finally, we display top matches

```
1 print(f"\nTop {k} matches:")
2 for rank, (i, s) in enumerate(zip(nn_idx, nn_score), 1):
3     r = META.iloc[i]
4     print(f"{rank:2d}.    domain={r.domain:6}    score={s:.3f}")
5     display(Image.open(r.image_path).resize((160,160)))
6 import random
7 for _ in range(3):
8     rand_idx = random.randrange(len(META))
9     show_retrieval(rand_idx, k=5)
```

2. This function, `def image_to_image_recall(k=1, chunk=2000)`, evaluates how well the model retrieves images from different domains within the top- k results. It's useful for assessing the model's ability to generalize beyond intra-domain similarity and ensure diversity in retrievals. The chunked processing ensures scalability for large datasets. It computes Recall@K for image-to-image retrieval, but with a cross-domain condition. It measures the percentage of images whose top- k nearest neighbors include at least one image from a different domain (`coco`, `sd`, `flickr`)

- We first initialize this process with `n = len(IMG_EMB)` number of images and `hits = np.zeros(n, dtype=bool)` which is a boolean-valued array to track whether each query image successfully found a cross-domain match in its top- k .
- We exclude self-matches with `np.fill_diagonal(sim[:, s:e], -np.inf)` which ensures an image isn't matched with itself by setting self-similarity in the current chunk to `-inf` and `sim` is computed from

```
1 for s in range(0, n, chunk):
2     e = min(s + chunk, n)
3     sim = img_norm[s:e] @ img_norm.T
```

- We find the top- k , check cross-domain, and return the metric with

```
1 topk = np.argpartition(-sim, k - 1, axis=1)[: , :k]
2 for row, candidates in enumerate(topk, s):
3     hits[row] = any(META.iloc[row].domain != META.iloc[c].domain
4                     for c in candidates)
4 return hits.mean() * 100
```

Here we find indices of top- k most similar images for each query, checks if at least one of these matches come from a different domain, and returns the the percentage of queries that found a valid cross-domain match.

- The output is Cross-domain Recall@1: 23.85 %, Cross-domain Recall@5: 54.85 %, Cross-domain Recall@10: 67.40 %.

The results indicate how well the model retrieves images from different domains in an image-to-image retrieval task. Specifically, a Cross-domain Recall@1 of 23.85%

means that for about one in four queries, the model’s top-ranked match came from a different domain than the query image. The Cross-domain Recall@5 of 54.85% shows that for over half of the queries, at least one of the top five retrieved images was cross-domain. Similarly, the Cross-domain Recall@10 of 67.40% indicates that nearly two-thirds of queries had a cross-domain match within the top ten results. These values suggest that while the model tends to retrieve images from the same domain more often, it still identifies meaningful cross-domain similarities, particularly as the number of considered neighbors increases. This can be a valuable metric for assessing how well the model generalizes visual similarity across different datasets or domains.

7 05_text_to_image.py

The `text_to_image.py` script is designed to load the outputs of a prior image-text retrieval experiment, perform qualitative and quantitative evaluations, and display the results. It begins by parsing command-line arguments to identify the experiment directory, the specific run folder (or defaulting to the latest available run), and the number of top- k matches to display. The script verifies that the selected run contains the necessary image and text embeddings, then loads and L2-normalizes them for cosine similarity computations. It sets up an OpenCLIP model to encode text queries, processes a set of example prompts, and computes similarity scores between each prompt and the image embeddings. The script prints the top- k most similar images for each query along with their domain, similarity score, and ID to allow qualitative inspection of retrieval results. Finally, it evaluates quantitative performance by computing text-to-image Recall@1, Recall@5, and Recall@10 using a memory-efficient, chunked routine, and prints these metrics to provide an overall measure of model performance on the retrieval task.

1. This code handles command-line argument parsing and experiment run selection for the retrieval.

- We create an argument parser to handle command-line inputs

```
1 parser = argparse.ArgumentParser()
2 parser.add_argument("--root", default=..., help=...)
3 parser.add_argument("--run", default=None, help=...)
4 parser.add_argument("--k", type=int, default=5, help=...)
5 args = parser.parse_args()
```

then we: add a `--root` argument to specify the experiments directory, add a `--k` argument to specify the exact run folder, add a `--k` argument to set how many top matches to display, and parse the arguments provided by the user at the command line.

2. We then check experiments directory to convert the root path to a `Path` object, add a helper function to verify embeddings if a folder contains both image and text embedding files to select the experiment run if it exists else exit with an error

```

1 EXP_ROOT = Path(args.root)
2 if not EXP_ROOT.exists():
3     sys.exit(f"Experiments dir not found: {EXP_ROOT}")
4 def has_embeddings(p: Path) -> bool:
5     return (p / "img_embs.npy").exists() and (p / "txt_embs.npy").
        exists()
6 if args.run:
7     RUN_DIR = EXP_ROOT / args.run
8     if not has_embeddings(RUN_DIR):
9         sys.exit(f"{RUN_DIR} lacks img_embs.npy / txt_embs.npy")
10 else:
11     runs = sorted((p for p in EXP_ROOT.iterdir() if has_embeddings(p)),
12                   key=lambda p: p.stat().st_mtime)
13     if not runs:
14         sys.exit(f"No runs with embeddings in {EXP_ROOT}")
15     RUN_DIR = runs[-1]

```

3. We load embeddings and metadata for retrieval tasks next

```

1 IMG_EMB = np.load(RUN_DIR / "img_embs.npy").astype("float32")
2 IDS      = json.loads((RUN_DIR / "ids.json").read_text())
3
4 PARQUET = Path(r"C:/Users/steph/OneDrive/Desktop/data/metadata.parquet"
5               )
6
7 META     = pd.read_parquet(PARQUET).set_index("id").loc[IDS]
8
9 img_norm = IMG_EMB / norm(IMG_EMB, axis=1, keepdims=True)
10 print(f"Loaded {IMG_EMB.shape[0]:,} image embeddings")

```

Here we load the saved image embeddings `img_embs.npy` from the selected experiment run directory to convert data to a `float32` type for consistent precision and compatibility then we load the local corresponding IDs used to map embeddings to metadata rows from a JSON file in the run directory so we can load the metadata and align with IDs. After that, we must L2 normalize embeddings so its length is 1.² This code sets up the image embeddings and metadata for retrieval evaluation by loading, aligning, and normalizing the data so that similarity computations can be performed efficiently and correctly.

4. This code sets up a text encoder and defines helper functions for text-to-image retrieval. It first determines the model type (e.g., ‘RN50’ or ‘ViT-B-32’) and selects the appropriate pretrained weights tag (‘openai’ or ‘laion2b_s34b_b79k’) based on the experiment run’s folder name. The OpenCLIP model is loaded on the CPU and switched to evaluation mode to disable gradient computation. The tokenizer is also retrieved. The `embed_text` function tokenizes an input string, generates its text embedding using the model, normalizes the embedding, and returns it as a NumPy array. The `topk_images` function computes the embedding of a query string, calculates cosine similarities with all image embeddings, and returns the indices and scores

²When performing reduction operations on an array or tensor, along a specific dimension `keepdims = True` determines whether the reduced dimension is retained in the output as a dimension of size 1.

of the top- k most similar images. The `print_results` function formats and prints these top matches, displaying each result's rank, domain, similarity score, and ID to provide a human-readable summary of retrieval performance for the query.

- We extract the model type ("RN50" or "ViT-B-32") from the run directory name and choose the correct pretrained weights tag based on the model

```
1 MODEL_NAME = RUN_DIR.name.split("_")[0]
2 PRETRAIN_TAG = "openai" if MODEL_NAME == "RN50" else "
    laion2b_s34b_b79k"
```

then we load the OpenCLIP model on the CPU with specified pretrained weights, switches the model to evaluation mode so no gradients are computed, and prepares the tokenizer function for text input

```
1 model, _, _ = open_clip.create_model_and_transforms(
2     MODEL_NAME, pretrained=PRETRAIN_TAG, device="cpu"
3 )
4 model.eval()
5 tokenize = open_clip.tokenize
```

- Now we do text embedding by tokenizing the input text. After that, we convert it to a NumPy array and L2-normalizes it for cosine similarity

```
1 def embed_text(text: str) -> np.ndarray:
2     with torch.no_grad():
3         tok = tokenize([text])
4         vec = model.encode_text(tok).cpu().numpy().astype("float32")
5     return vec / norm(vec, axis=1, keepdims=True)
```

- We are ready to get the top- k image retrieval which embeds the query text, computes the cosine similarity with all image embeddings, and efficiently find the indices of the top- k most similar images

```
1 def topk_images(query: str, k: int):
2     q_vec = embed_text(query)[0]
3     sim = q_vec @ img_norm.T
4     idx = np.argpartition(-sim, k)[:k]
5     idx = idx[np.argsort(-sim[idx])]
6     return idx, sim[idx]
7 def print_results(query: str, k: int):
8     idx, scores = topk_images(query, k)
9     print(f"\nQuery: \"{query}\"")
10    for rank, (i, s) in enumerate(zip(idx, scores), 1):
11        row = META.iloc[i]
12        print(f"{rank:2d}.    domain={row.domain:6}    score={s:.3f}
            id={row.name}")
```

At the end, we display the query and lists each retrieved image's rank, domain, similarity score, and ID. The output is

```

Query: "a red sports car on a city street"
1. domain=sd      score=0.245 id=sd_53939
2. domain=coco    score=0.214 id=co_425248
3. domain=coco    score=0.208 id=co_651152
4. domain=coco    score=0.208 id=co_757318
5. domain=coco    score=0.206 id=co_303415

Query: "two dogs playing in the snow"
1. domain=coco    score=0.250 id=co_128300
2. domain=flickr  score=0.232 id=fl_64289
3. domain=coco    score=0.229 id=co_549738
4. domain=coco    score=0.227 id=co_496420
5. domain=flickr  score=0.208 id=fl_67871

Query: "cyberpunk city at night with neon lights"
1. domain=coco    score=0.193 id=co_769416
2. domain=coco    score=0.193 id=co_705543
3. domain=sd      score=0.191 id=sd_13997
4. domain=coco    score=0.185 id=co_452509
5. domain=sd      score=0.172 id=sd_76838

Recall@K on this run:
Recall@1: 48.15 %
Recall@5: 76.50 %
Recall@10: 85.20 %

```

Figure 2. Qualitative retrieval (top-5 results per query).

These results indicate solid performance: the model retrieves the correct image in the top-1 match nearly half the time, and in the top-10 match most of the time. The individual queries demonstrate that the model can surface reasonable and cross-domain matches, reflecting meaningful alignment between text and image embeddings.

8 06_evaluation_visualization

The code implements a complete pipeline for loading, organizing, and comparing the results of multiple retrieval experiment runs. It begins by iterating over experiment directories, loading configuration files and metric outputs such as `config.json`, `metrics.json`, and `image_image_metrics.json`. These records are aggregated into a DataFrame, where each row represents an experiment run with its parameters and retrieval metrics (e.g., Recall@1, Recall@5, Recall@10). The DataFrame is cleaned and enriched with helper columns such as the total number of samples and a human-readable label combining model name and run ID. The code then generates plots to visualize performance. A line plot shows how Recall@K values change with sample size, helping to illustrate scaling behavior. A radar chart compares multiple runs across different retrieval metrics, making it easy to observe trade-offs between models. Finally, a dual-axis bar plot compares model size in terms of parameters and embedding time, providing insight into computational efficiency versus model complexity. This workflow supports systematic evaluation of retrieval experiments and facilitates clear reporting of results.

9 07_dataset_to_dataset.ipynb

The code loads embeddings, metadata, and experiment configuration for a specific run directory and computes cross-dataset retrieval metrics between COCO and Flickr captions. It begins by loading image and text embeddings, ID mappings, and associated metadata from the specified Parquet file, then aligns and normalizes the text embeddings using L2 normalization. The domain distribution is printed for reference. The code identifies indices of COCO and Flickr captions and extracts their corresponding embeddings

```
1 idx_coco      = META.index[META["domain"] == "coco"]
2 idx_flickr    = META.index[META["domain"] == "flickr"]
3
4 pos_coco      = META.index.get_indexer(idx_coco)
5 pos_flickr    = META.index.get_indexer(idx_flickr)
6
7 coco_vecs     = txt_norm[pos_coco]
8 flickr_vecs   = txt_norm[pos_flickr]
9
10 def recall_cross(source, target, k=1, chunk=1000):
11     n = source.shape[0]
12     hits = 0
13     for s in range(0, n, chunk):
14         e = min(s + chunk, n)
15         sim = source[s:e] @ target.T
16         topk = np.argpartition(-sim, k-1, axis=1)[:k]
17         hits += np.sum(topk == np.arange(e - s)[:k], None)
18     return hits / n * 100.0
19
20 metrics_dd = {
21     "coco_flickr_R@1": round(recall_cross(coco_vecs, flickr_vecs, 1), 2)
22     ,
23     "flickr_coco_R@1": round(recall_cross(flickr_vecs, coco_vecs, 1), 2)
24     ,
25 }
```

A function is defined to compute Recall@1 for cross-dataset retrieval using chunked similarity computations for efficiency. It calculates how often a caption from one dataset retrieves its correct cross-dataset match within the top-1 nearest neighbors. The results for COCO-to-Flickr and Flickr-to-COCO retrieval are rounded and saved as JSON. This workflow evaluates how well the model aligns captions across domains and records the outcomes for further analysis.

10 08_model_ablation.ipynb

The code loads and aggregates retrieval metrics from experiment runs in the `experiments` directory to compare model performance across different settings or architectures. It defines a function to load configuration and metric files for each run, then constructs a DataFrame that contains information such as the model type, pretrained weights, sample size, and Recall@K metrics. The script generates bar plots to visualize Recall@1, Recall@5, and

Recall@10 across different runs, with the x -axis showing run identifiers and the y -axis representing retrieval accuracy in percentage. The plots make it easy to compare retrieval performance of different models or configurations at a glance. In this example, results from a single run are shown, demonstrating the pipeline’s capability for model comparison and visualization of key retrieval metrics.

```

1 # Compare retrieval performance across different model architectures or
  settings
2 def load_run(p: Path):
3     out = {"folder": p.name}
4     for f in ["config.json", "metrics.json", "image_text_metrics.json"]:
5         path = p / f
6         if path.exists():
7             out.update(json.loads(path.read_text()))
8     return out
9 # Load all runs that contain valid metrics
10 records = []
11 for p in EXP_ROOT.iterdir():
12     if p.is_dir() and (p / "image_text_metrics.json").exists():
13         run = load_run(p)
14         records.append(run)
15
16 runs_df = pd.DataFrame(records).sort_values("folder").reset_index(drop=True)
17 cols = ["folder", "model", "pretrained", "max_samples", "R@1", "R@5", "R@10"]
18 df = runs_df[cols].copy()
19 print(df)
20 # Plot Recall@1 comparison
21 plt.figure(figsize=(8, 4))
22 plt.bar(df["folder"], df["R@1"])
23 plt.xticks(rotation=75, ha="right")
24 plt.ylabel("Recall@1 (%)")
25 plt.title("Model comparison: Recall@1")
26 plt.tight_layout()
27 plt.show()
28 # Plot Recall@5 and Recall@10
29 for recall in ["R@5", "R@10"]:
30     plt.figure(figsize=(8, 4))
31     plt.bar(df["folder"], df[recall])
32     plt.xticks(rotation=75, ha="right")
33     plt.ylabel(f"{recall} (%)")
34     plt.title(f"Model comparison: {recall}")
35     plt.tight_layout()
36     plt.show()

```

11 09_visual_nn_demo

The code demonstrates a qualitative image-to-image retrieval evaluation using cosine similarity between image embeddings. It begins by loading precomputed image embeddings and corresponding metadata, normalizing the embeddings to enable cosine similarity via

dot product. Five random query images are selected, and for each, the model identifies and displays the top-5 nearest neighbor images from the dataset. The results are visualized using matplotlib, where each row shows the query image followed by its top matches, along with their domains (e.g., COCO, Flickr, Stable Diffusion) and similarity scores. This visualization provides an intuitive way to assess the model’s ability to retrieve semantically similar images across different domains, helping users verify qualitative performance beyond numerical metrics.

```

1 N_QUERIES = 5
2 random.seed(42)
3 query_indices = random.sample(range(img_norm.shape[0]), k=N_QUERIES)
4 K = 5 # neighbours to display
5 # Visualise neighbours for each query
6 for qi in query_indices:
7     q_vec = img_norm[qi]
8     sim = q_vec @ img_norm.T
9     sim[qi] = -1.0
10    nn_idx = np.argpartition(-sim, K)[:K]
11    nn_idx = nn_idx[np.argsort(-sim[nn_idx])] # sort by similarity
12    fig, axes = plt.subplots(1, K + 1, figsize=(2.5*(K+1), 3))
13    fig.suptitle(f"Query idx {qi} domain: {META.iloc[qi].domain}")
14    axes[0].imshow(open_image(META.iloc[qi]))
15    axes[0].set_title("Query", fontsize=10)
16    axes[0].axis("off")
17    # show neighbours
18    for rank, idx in enumerate(nn_idx, 1):
19        row = META.iloc[idx]
20        axes[rank].imshow(open_image(row))
21        axes[rank].set_title(f"{rank}. {row.domain}\n{sim[idx]:.3f}",
22                             fontsize=9)
23        axes[rank].axis("off")
24    plt.tight_layout()
25    plt.show()

```

12 10_Run_Experiment

This code defines a complete pipeline for embedding image-text pairs using a chosen OpenCLIP backbone model and evaluating retrieval performance via Recall@K. It begins by setting configuration parameters such as model architecture, device preset, batch size, and maximum sample size. Metadata and images are loaded, and a dataset is constructed where each sample consists of an image and its associated caption, with preprocessing applied through standard resizing, cropping, and normalization transforms. The OpenCLIP model is loaded onto the specified device and set to evaluation mode. The pipeline processes the data in batches, computing normalized embeddings for both images and text. These embeddings are used to compute cosine similarities and Recall@1, Recall@5, and Recall@10 metrics, quantifying how often the correct image is retrieved within the top-k predictions. Finally, the embeddings, metrics, and configuration details (including model

size and embedding time) are saved to an output directory for reproducibility and later analysis.

The results are reasonably good, showing that the model retrieves the correct image as the top-1 match 43.09% of the time and within the top-10 matches 75.29% of the time, which reflects solid but not exceptional retrieval performance.